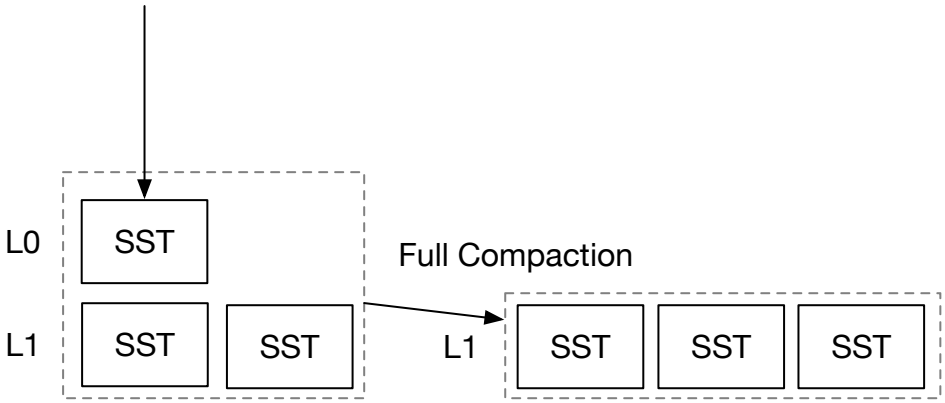
# Compaction Implementation




In this chapter, you will:

- Implement the compaction logic that combines some files and produces new files.
- Implement the logic to update the LSM states and manage SST files on the filesystem.
- Update LSM read path to incorporate the LSM levels.

To copy the test cases into the starter code and run them,

```
cargo x copy-test --week 2 --day 1
cargo x scheck
```

> It might be helpful to take a look at week 2 overview before reading this chapter to have a general overview of compactions.

## Task 1: Compaction Implementation

In this task, you will implement the core logic of doing a compaction – merge sort a set of SST files into a sorted run. You will need to modify:

```
src/compact.rs
```

Specifically, the `force_full_compaction` and `compact` function. `force_full_compaction` is the compaction trigger that decides which files to compact and update the LSM state. `compact` does the actual compaction job that merges some SST files and return a set of new SST files.

Your compaction implementation should take all SSTs in the storage engine, do a merge over them by using `MergeIterator`, and then use the SST builder to write the result into new files. You will need to split the SST files if the file is too large. After compaction completes, you can update the LSM state to add all the new sorted run to the first level of the LSM tree. And, you will need to remove unused files in the LSM tree. In your

implementation, your SSTs should only be stored in two places: the L0 SSTs and the L1 SSTs. That is to say, the `levels` structure in the LSM state should only have one vector. In `LsmStorageState`, we have already initialized the LSM to have L1 in `levels` field.

Compaction should not block L0 flush, and therefore you should not take the state lock when merging the files. You should only take the state lock at the end of the compaction process when you update the LSM state, and release the lock right after finishing modifying the states.

You can assume that the user will ensure there is only one compaction going on. `force_full_compaction` will be called in only one thread at any time. The SSTs being put in the level 1 should be sorted by their first key and should not have overlapping key ranges.

▶ Spoilers: Compaction Pseudo Code

In your compaction implementation, you only need to handle `FullCompaction` for now, where the task information contains the SSTs that you will need to compact. You will also need to ensure the order of the SSTs are correct so that the latest version of a key will be put into the new SST.

Because we always compact all SSTs, if we find multiple version of a key, we can simply retain the latest one. If the latest version is a delete marker, we do not need to keep it in the produced SST files. This does not apply for the compaction strategies in the next few chapters.

There are some things that you might need to think about.

- How does your implementation handle L0 flush in parallel with compaction? (Not taking the state lock when doing the compaction, and also need to consider new L0 files produced when compaction is going on.)
- If your implementation removes the original SST files immediately after the compaction completes, will it cause problems in your system? (Generally no on macOS/Linux because the OS will not actually remove the file until no file handle is being held.)

# Task 2: Concat Iterator

In this task, you will need to modify,

```
src/iterators/concat_iterator.rs
```

Now that you have created sorted runs in your system, it is possible to do a simple optimization over the read path. You do not always need to create merge iterators for your SSTs. If SSTs belong to one sorted run, you can create a concat iterator that simply iterates the keys in each SST in order, because SSTs in one sorted run do not contain overlapping

key ranges and they are sorted by their first key. We do not want to create all SST iterators in advance (because it will lead to one block read), and therefore we only store SST objects in this iterator.

## Task 3: Integrate with the Read Path

In this task, you will need to modify,

```
src/lsm_iterator.rs
src/lsm_storage.rs
src/compact.rs
```

Now that we have the two-level structure for your LSM tree, and you can change your read path to use the new concat iterator to optimize the read path.

You will need to change the inner iterator type of the `LsmStorageIterator`. After that, you can construct a two merge iterator that merges memtables and L0 SSTs, and another merge iterator that merges that iterator with the L1 concat iterator.

You can also change your compaction implementation to leverage the concat iterator.

You will need to implement `num_active_iterators` for concat iterator so that the test case can test if concat iterators are being used by your implementation, and it should always be 1.

To test your implementation interactively,

```
cargo run --bin mini-lsm-cli-ref -- --compaction none # reference solution
cargo run --bin mini-lsm-cli -- --compaction none # your solution
```

And then,

```
fill 1000 3000
flush
fill 1000 3000
flush
full_compaction
fill 1000 3000
flush
full_compaction
get 2333
scan 2000 2333
```

# Test Your Understanding

- What are the definitions of read/write/space amplifications? (This is covered in the overview chapter)
- What are the ways to accurately compute the read/write/space amplifications, and what are the ways to estimate them?
- Is it correct that a key will take some storage space even if a user requests to delete it?
- Given that compaction takes a lot of write bandwidth and read bandwidth and may interfere with foreground operations, it is a good idea to postpone compaction when there are large write flow. It is even beneficial to stop/pause existing compaction tasks in this situation. What do you think of this idea? (Read the SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores paper!)
- Is it a good idea to use/fill the block cache for compactions? Or is it better to fully bypass the block cache when compaction?
- Does it make sense to have a `struct ConcatIterator<I: StorageIterator>` in the system?
- Some researchers/engineers propose to offload compaction to a remote server or a serverless lambda function. What are the benefits, and what might be the potential challenges and performance impacts of doing remote compaction? (Think of the point when a compaction completes and what happens to the block cache on the next read request...)

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.